



OPERATIONAL SYSTEM ARCHITECTURE:

ARFOM-DB (CROW'S FOOT)

CROW'S FOOT •
10 TABLES

10 Core Relational Tables, Airframe Topology, Dispatch, Passenger Ticketing & Revenue Ledger

Infrastructure (4)

airports, routes, aircraft, seats

Operations (3)

Commercial (3)

Audit & Finance (2)

DOMAIN MODULES: ■ Physical Infrastructure (Airports, Routes, Aircraft, Seats)

■ Operations & Dispatch (Flights, Checkins, Baggage)

■ Commercial & Customer (Passengers, Bookings, Tickets)

■ Settlement & Audit (Payments, Cancellations)

Visual Entity-Relationship (ER) Schema

VECTOR SCHEMA CARDINALITY FLOW (FULL ARCHITECTURE)

100% Vector
SVG



RELATIONAL SCHEMA DESIGN: INFRASTRUCTURE & FLEET

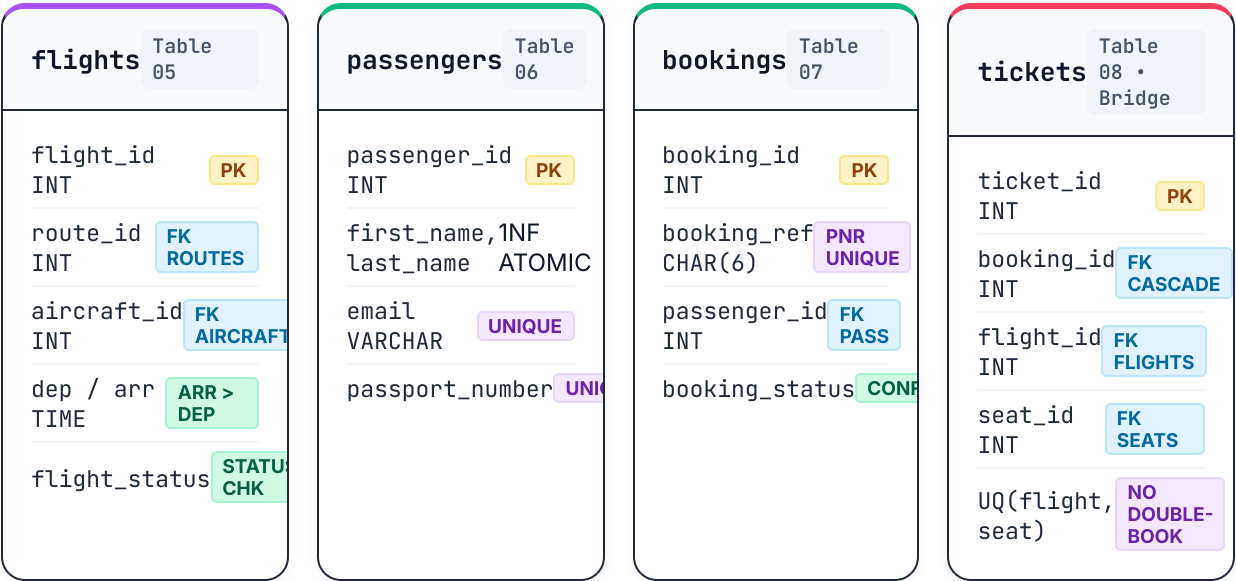
MODULE 01 /
03

DDL Definitions, Foreign Key Cascades, Capacity & Seat Invariants

airports Table 01 airport_code CHAR(3) PK airport_name NOT VARCHARNULL city NOT VARCHARNULL country NOT VARCHARNULL	routes Table 02 route_id INT PK origin_airport FK RESTRICT dest_airport FK RESTRICT distance_km > 0 CHK: origin $\diamond$ CHECK dest	aircraft Table 03 aircraft_id INT PK model NOT VARCHARNULL total_capacity > 0	seats Table 04 seat_id INT PK aircraft_id INT FK CASCADE seat_number 12A, VARCHARNULL 14B seat_class CLASS CHK UQ(aircraft, seat) UNIQUE
--	--	---	---

RELATIONAL SCHEMA DESIGN: PASSENGERS & FLIGHT TICKETING

Flight Inventory, Commercial Reservations (PNR) & Double-Booking Invariants



RELATIONAL SCHEMA DESIGN: DEPARTURE CONTROL & AUDIT

1:1 Check-in Enforcements, Baggage Weights, Payments & Bounded Refund Rules

checkins	Table 09	
checkin_id	INT	PK
ticket_id	INT	1:1 UNIQUE FK
boarding_pass		UNIQUE TOKEN
checkin_time	TIMESTAMP	

baggage	Table 10	
baggage_id	INT	PK
checkin_id	INT	FK CASCADE
weight_kg		>= 0.00
excess_fee		>= 0.00

payments	Table 11	
payment_id	INT	PK
booking_id	INT	FK BOOKINGS
amount_paid	> 0.00	
payment_method	UPI/C	

cancellations		Tab 12
cancellation_id	INT	PK
ticket_id	INT	1:1 UNIQUE FK
refund_amount		>= 0.00
TRG: refund ≤ fare		BOUND TRG

MATHEMATICAL NORMALIZATION PROOF: UNF → 3NF → BCNF

BCNF
COMPLIANT

Elimination of Partial & Transitive Dependencies with Lossless Join Decomposition

1. First Normal Form (1NF)

- Eliminated multi-valued attribute arrays (e.g. composite passenger names split into `first_name` & `last_name`).
- Every column holds strictly atomic values.

2. Second Normal Form (2NF)

- Eliminated Partial Functional Dependencies where non-prime attributes depended on a subset of composite keys.
- Aircraft model separated from seats:
`aircraft_id` → `model`.

3. Third Normal Form (3NF)

- Eliminated Transitive Dependencies: $X \rightarrow Y \rightarrow Z$.
- Airports separated from routes so `city/country` depend directly on `airport_code`, not transitively on `route_id`.

4. Boyce-Codd Form (BCNF)

- For every non-trivial functional dependency $X \rightarrow Y$, X is strictly a **Superkey**.
- Preserves lossless join property: $R1 \bowtie R2 \rightarrow R1$ or $R2$.

THE 5 MANDATORY BUSINESS RULES & LIVE NEGATIVE TESTS

ACTIVE INVARIANTS

Declarative SQL Constraints & Trigger Invariants Intercepting Illegal Operations

RULE #	MANDATORY BUSINESS RULE	ENFORCEMENT MECHANISM	NEGATIVE TEST SQL STATEMENT	LIVE ENGINE ERROR INTERCEPTION
RULE 1	One seat per passenger per flight	UNIQUE (flight_id, seat_id)	INSERT INTO tickets ... (flight 101, seat 1)	UNIQUE constraint failed: tickets.flight_id, tickets.seat_id
RULE 2	Aircraft-Capacity limits	trg_verify_aircraft_capacity	INSERT 5th ticket on 4-seat Boeing 737	Capacity Overrun: Airframe capacity reached for this flight
RULE 3	Non-negative baggage	CHECK (weight_kg >= 0.00)	INSERT INTO baggage ... (weight: -5.50 kg)	CHECK constraint failed: weight_kg ≥ 0.00
RULE 4	Check-in for confirmed bookings	trg_verify_checkin_eligibility	INSERT checkin on CANCELLED ticket	Check-in Denied: Booking must be CONFIRMED and ticket ISSUED
RULE 5	Strict refund rules (Refund ≤ Fare)	trg_enforce_refund_bounds	INSERT refund ₹10,000 on ₹8,500 ticket	Accounting Anomaly: Refund exceeds ticket base fare

ANALYTICAL SQL REPORTS & OPERATIONAL VIEWS

7 RUBRIC
QUERIES

Demonstrating Multi-Table Joins, Scalar Nested Subqueries, Aggregations & Yield Metrics

1. Real-Time Flight Manifest (View + 6 Joins)

VIEW

```
SELECT flight_id, corridor,  
       seat_number, passenger,  
       boarding_pass  
FROM view_master_manifest LIMIT 3;
```

2. Occupancy & Efficiency (Count + CASE)

AGGREGATE

```
SELECT flight_id, airframe,  
       occupancy_pct,  
       operational_efficiency  
FROM view_flight_occupancy ORDER  
BY occupancy_pct DESC;
```

3. Route Demand (Corridors & Gross Yield)

GROUP
BY

```
SELECT corridor, flights_operated,  
       total_revenue  
FROM routes r LEFT JOIN flights f  
ON r.route_id = f.route_id GROUP  
BY r.route_id;
```

4. On-Time Performance (Nested Subquery)

SUBQUERY

```
SELECT flight_status, count(*),  
       ROUND(count(*) * 100.0 / (SELECT  
count(*) FROM flights), 1) || '%'  
AS pct  
FROM flights GROUP BY  
flight_status;
```

VIVA DEFENSE STRATEGY & LIVE DEMONSTRATION

DEFENSE
DOSSIER

Emergency 15-Second Defense Recitals & Direct Prototype Access

Q: How do you enforce 1:1 vs 1:N in relational schema?

"A foreign key is 1:N by default. To make it strictly 1:1, we place a **UNIQUE constraint** on the FK column. For example, `checkins.ticket_id` is marked UNIQUE, guaranteeing at most one check-in per ticket."

Q: Why did we decompose Passengers & Flights?

"Passengers and Flights have a Many-to-Many (M:N) relationship. To satisfy 1NF, we decomposed M:N into two 1:N relationships via the junction entity `tickets`."



Live Query Workbench

Instant SQL query execution on 60+ rows with active constraint validation

 **Launch Live Workbench**

